



软件工程的基本概念

软件

构成

- 程序 程序：独唱或小合唱
软件：万人大大合唱
- 数据
- 文档

四大特征

- 复杂性
- 不可见性
- 易变性 变化是永久的主题
无法被制造
- 一致性

软件的发展

- 第一阶段 — 主要用于【数值计算】
- 第二阶段 — 开始向【商务领域】推广，出现了数据库、结构化编程等技术
- 第三阶段 — 开始关注对软件开发过程的【管理】及【工程性】的开发
- 第四阶段 — Internet和Web → 分布式、异构环境下的软件
- 第五阶段 — 软件的服务化、系统互操作的需求

开发方法的发展

- 早期开发 软件 = 程序
面对过程的软件 = 数据结构 + 算法
- 结构化开发 每个部分的结构明确，比如主函数、函数、算法
软件 = 程序 + 数据 + 文档
图示
- 面向对象的方法 对象 = 函数 + 数据
软件 = 对象 + 消息
图示
- 构件化方法 构件 = 对象 + 消息 or 构件 = 实体 + 接口
软件 = 构件 + 连接件 or 软件 = 构件 + 框架
图示
- SOA方法 服务总线ESB — 控制并切换<服务1(需求)>是调用构件A的服务1，还是构件B的服务1
软件 = 服务 + 消息 + 总线
服务 = 构件的公共标准接口
图示
- 云计算方法

软件工程

历史渊源

- 没有银弹 复杂的软件工程问题无法靠简单的答案来解决！
在软件开发过程里是没有什么【万能武器】，只有各种方法综合运用，才是解决之道
- 软件危机 计算机软件的开发和维护过程所遇到的一系列严重问题

定义

- 定义1
 - 定义2
 - 定义3
- 软件工程重要不是技术而是解决问题的思想

软件生命周期

软件定义

- 三个阶段 问题定义
可行性研究
需求分析

软件开发

- 解决【如何做】的问题
- 四个阶段 概要设计
详细设计
编码
测试

软件维护

- 改正性维护 也就是诊断和改正在使用过程中发现的【软件错误】
- 适应性维护 即修改软件以适应【环境的变化】
- 完善性维护 即根据【用户的要求】改进或【扩充软件】使它更完善
- 预防性维护 即修改软件为【将来的维护】活动预先做准备

软件开发中的角色构成

客户

- 决策者
- 终端用户 — 易用性
- 系统管理员

软件公司

- 决策者
- 软件销售与市场人员
- 咨询师、需求分析师
- 软件架构师、软件设计师 — 体系结构的设计与实现
- 开发人员 开发经理
编程人员 — 静态软件模块(源代码、数据文件)的组织与管理
系统集成人员 — 运行时性能、可扩展性、吞吐量等
系统部署人员 — 运行时系统拓扑、安装、通信等
- 维护人员

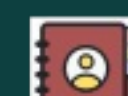
软件工程的核心思想

核心概念模型

- 问题空间 需求分析方法
- 需求模型 系统设计方法
- 设计模型 程序设计方法
- 实现模型 部署与维护方法
- 部署与运行模型
- 解空间

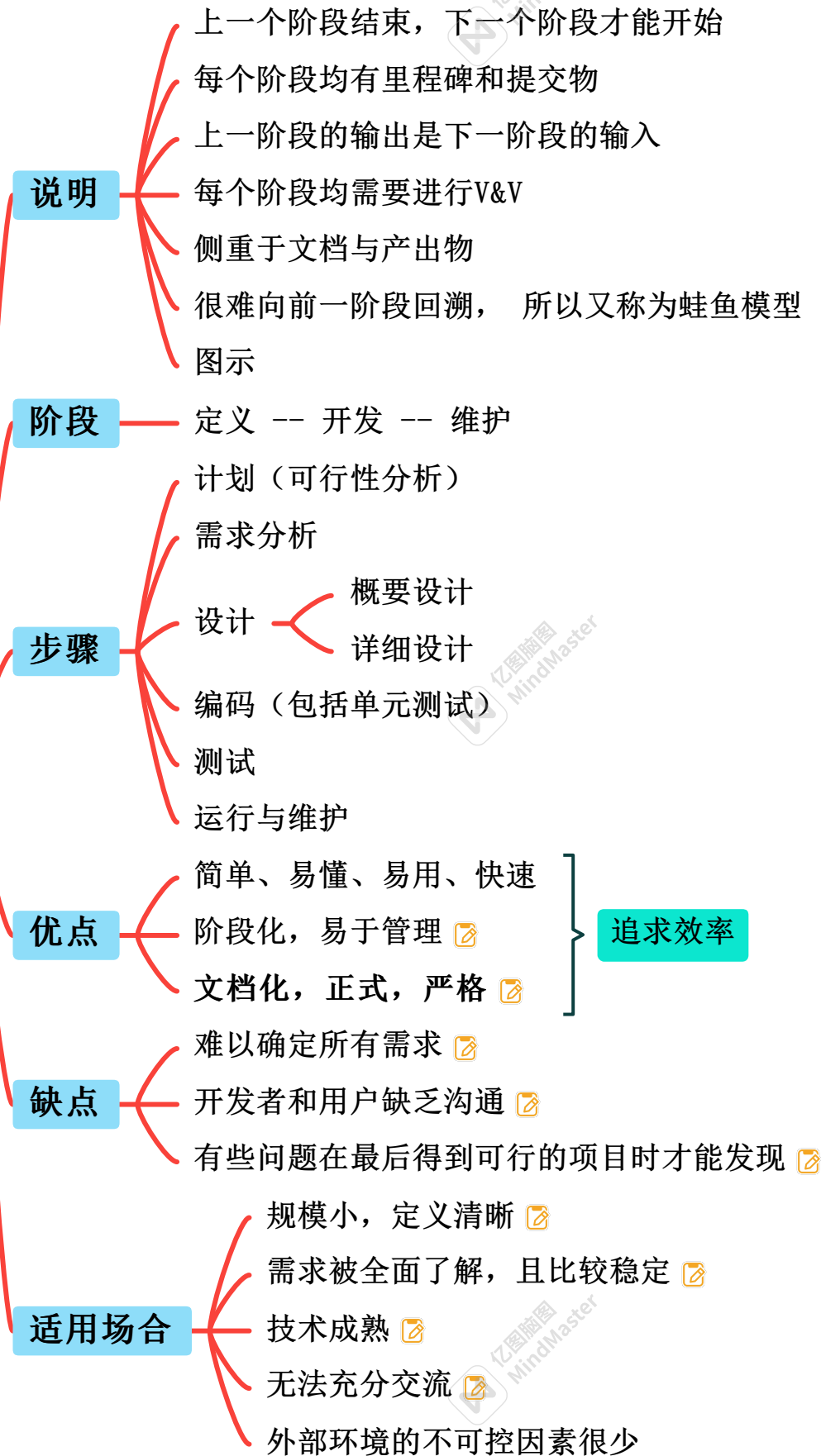
核心思想方法

- 分而治之
- 复用
- 折中
- 演化

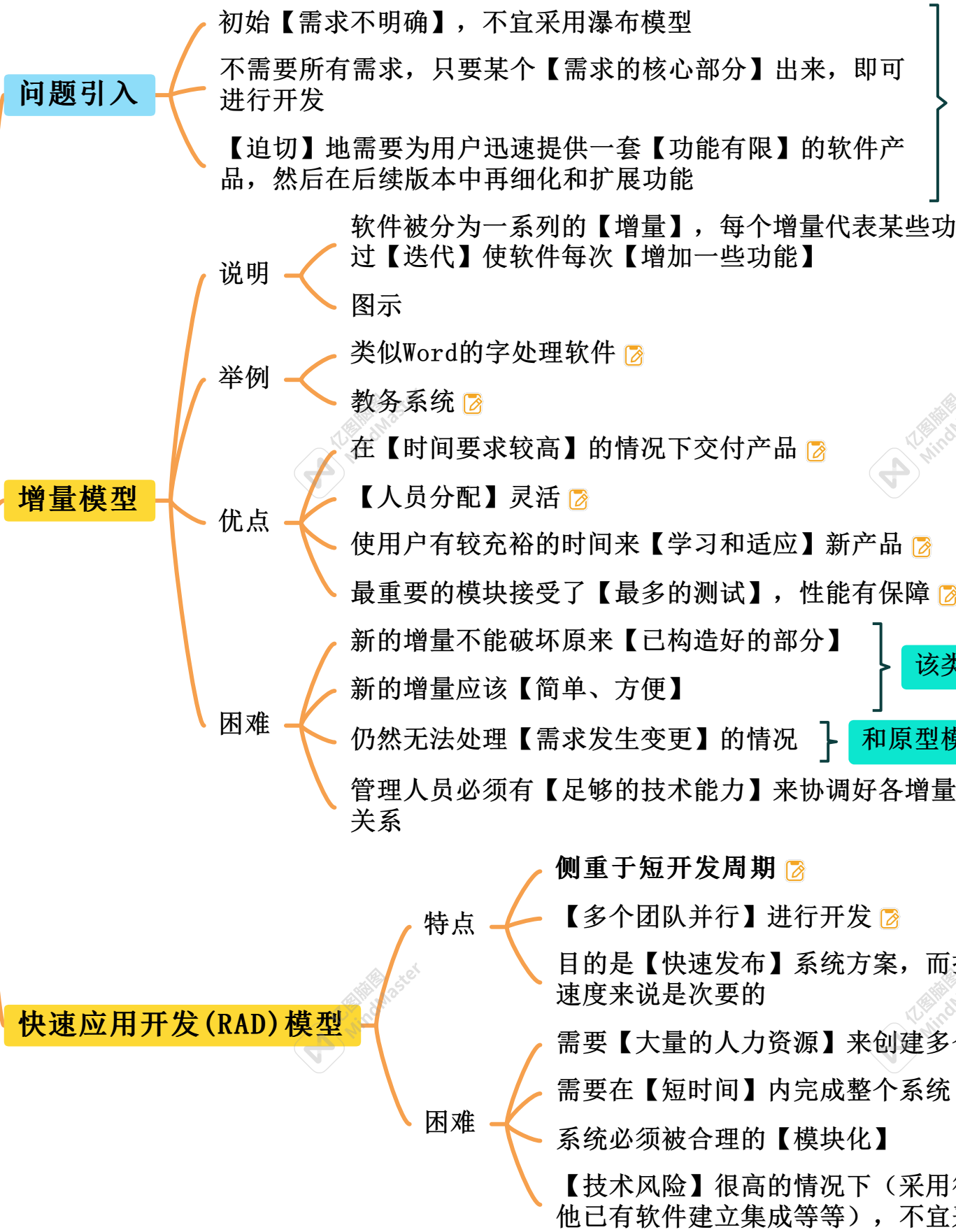


软件生命周期模型

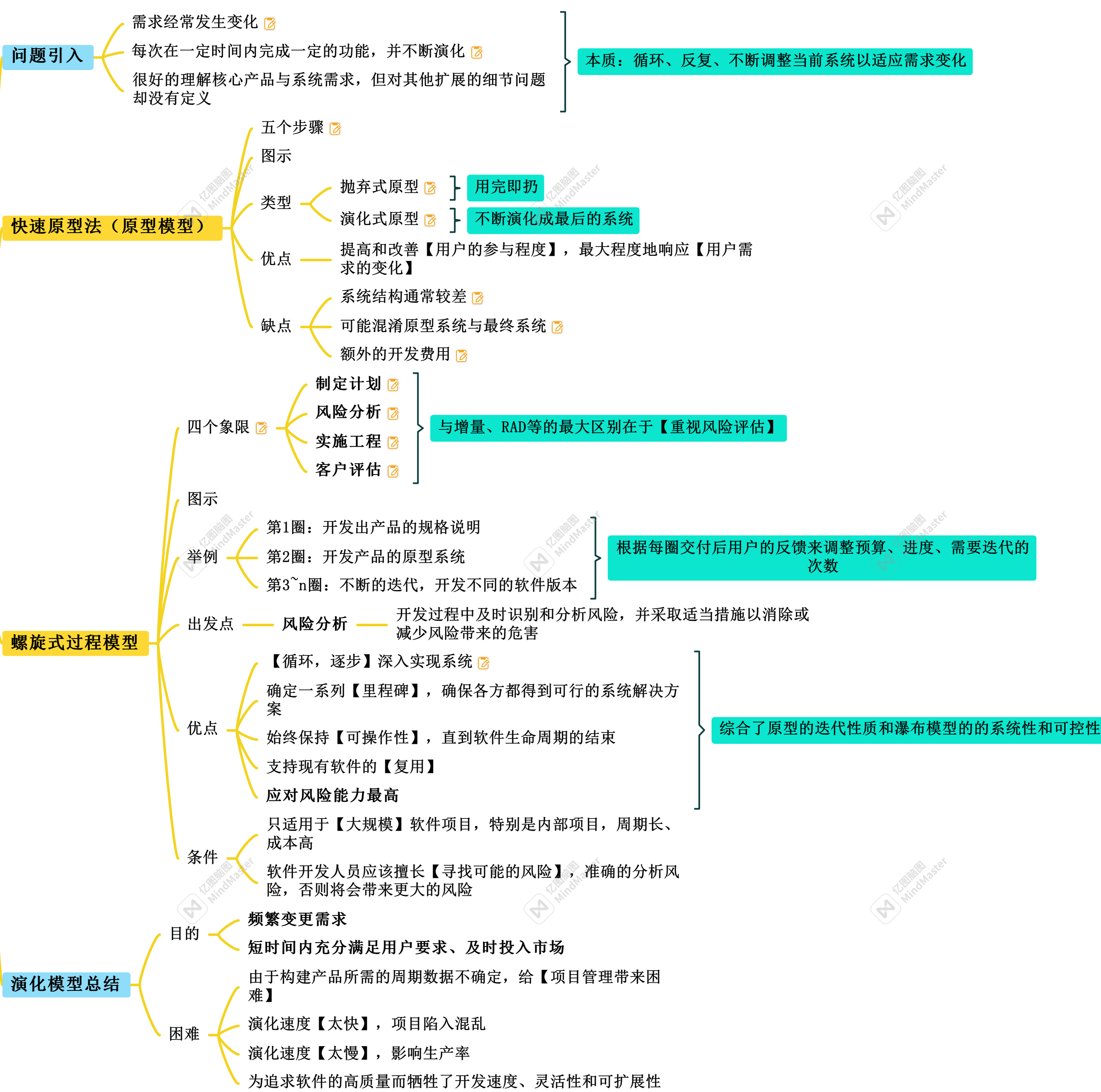
瀑布模型



增量过程模型



演化过程模型

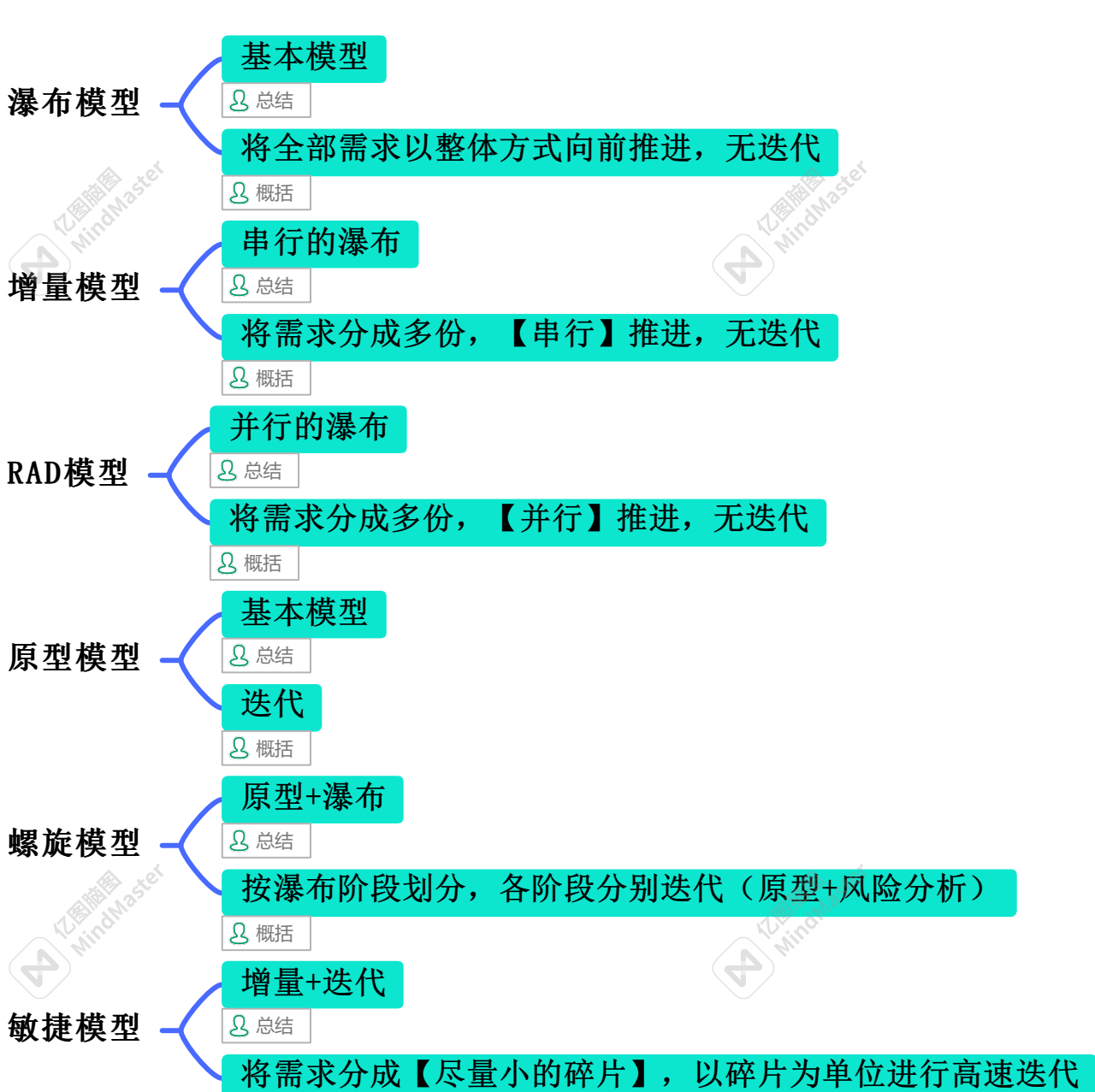


敏捷方法与过程模型



归纳总结

一句话概括各种模型





软件项目管理

基本概念

项目

项目管理 (PM)

软件项目的特征

- 软件产品的不可见性：软件项目复杂和抽象
- 项目的高度不确定性：预定计划与实际情况存在较大偏差
- 软件过程的多变化性：不确定、不稳定
- 软件人员的高技能及其高流动性：风险

人员 (People)

参与人员

- 高级管理者
- 项目 (技术) 管理者
- 开发人员
- 客户
- 最终用户

举例

开发团队

好坏决定于

- 项目经理的管理风格
- 团队人员的数量和技能水平
- 项目的总体难易程度

组件团队时考虑

- 项目方面
- 人员方面

组织方式

- 一窝蜂模型
- 主治医师模式
- 社区模式
- 交响乐模式
- 爵士乐模式
- 功能团队模式
- 官僚模式

4P

产品 (Product)

- 各阶段不同的产品
- 首选应确定软件范围
- 产品分解结构

功能特性

一个成功的软件必须具备的

非功能特性

反映一个软件好坏的：性能、可用性、安全、法律等

过程 (process)

- 步骤
- 工作结构分解

选择合适的软件过程模型

不同过程模型适用不同类型的软件项目

根据所选的过程模型，对其进行适应性修改

确定过程中应包含的工作任务列表

项目 (project)

四个方面

- 范围 (Scope)
- 时间 (Time)
- 成本 (Cost)
- 质量 (Quality)

主要任务

- 项目可行性分析与估算
- 项目进度安排
- 项目风险管理
- 项目质量管理
- 项目跟踪与控制

W5HH原则

- Why 为什么要开发这个系统？
- What 将要做什么？
- When 什么时候做？
- Who 某功能由谁来做？
- Where 他们的机构组织位于何处？
- How 如何完成技术与管理工作？
- How much 各种资源分别需要多少？

可行性分析与估算

开始之前预先估计

- 工作量大小
- 需要的时间
- 需要的人员数量
- 所需的资源 (软件, 硬件)
- 风险

得出项目的可行性

确定范围

什么是软件的范围

- Needs: 客户/最终用户的请求、想法和业务需求
- Requirements: 未来系统所应具备的功能的
- Exclusions: 将不包含在未来系统中的功能
- Baseline: 未来系统中应包含的功能
- 图示

如何确定

并不是用户说什么就做什么

可行性分析

- 技术可行性
- 经济可行性
- 时间可行性
- 资源可行性

项目规模估算

- 估算方法 (公式)
- 实际的成本往往比估算的大得多

软件项目规划

进度计划与监控

甘特图

分配资源

- 资金
- 人员
- 设备
- 环境

明确产出结果

每一项任务的产出结果是什么？对应于PBS中的哪一部分？

明确里程碑

项目的关键产出物，标志着某一阶段的完成

项目进度跟踪

- 为何要跟踪项目进度？
- 如何跟踪 (方法)

项目风险管理

风险类型

- 软件规模风险
- 商业影响风险
- 客户相关风险
- 软件过程风险
- 开发环境风险
- 开发技术风险
- 开发人员风险

步骤

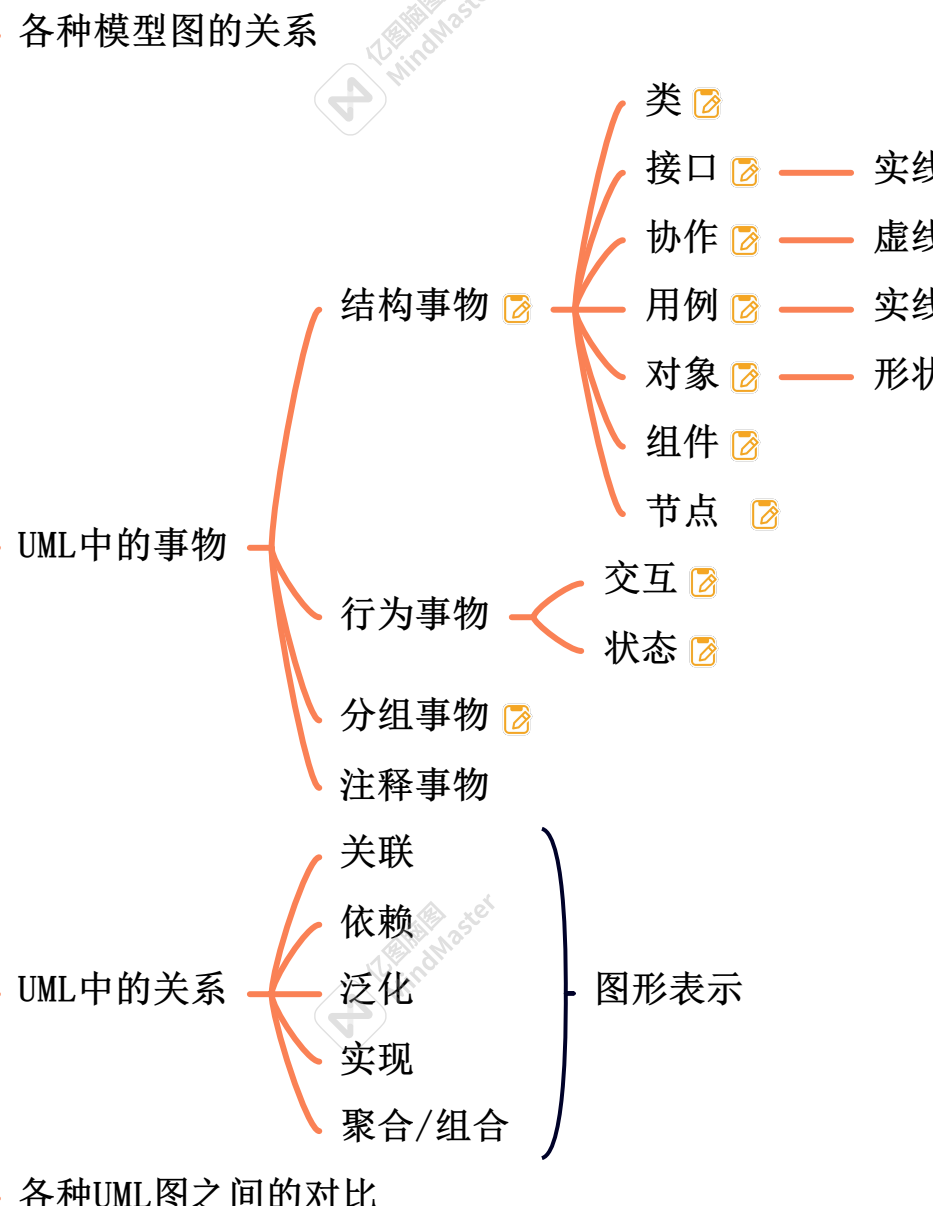
- 列出可能的风险
- 估计风险发生的可能性或概率
- 建立风险表
- 估计风险可能产生的影响或后果
- 风险求精
- 风险环节、监测和管理
- 风险应急计划

预测与分析

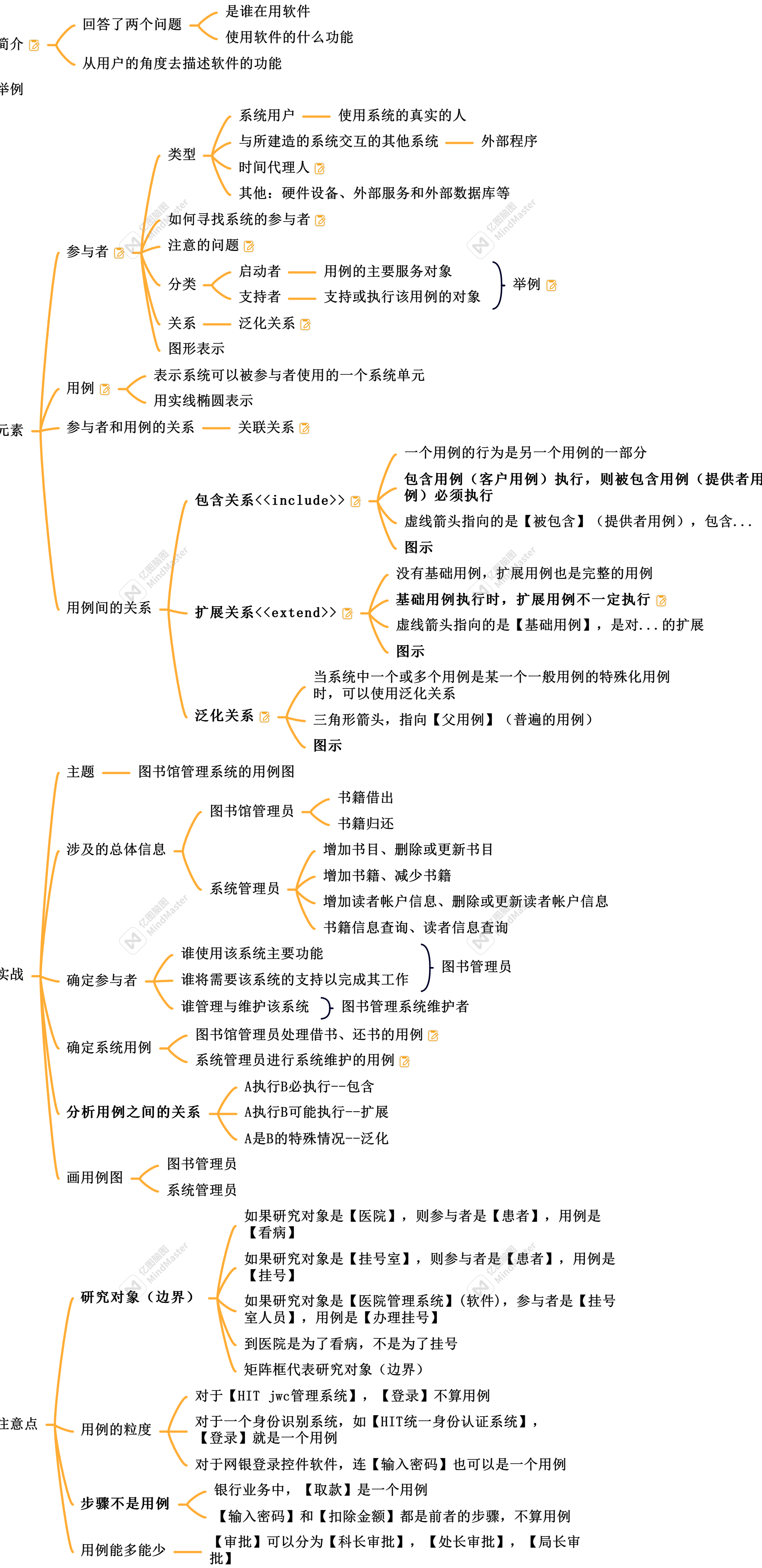
其他方面

UML(上)

UML元素



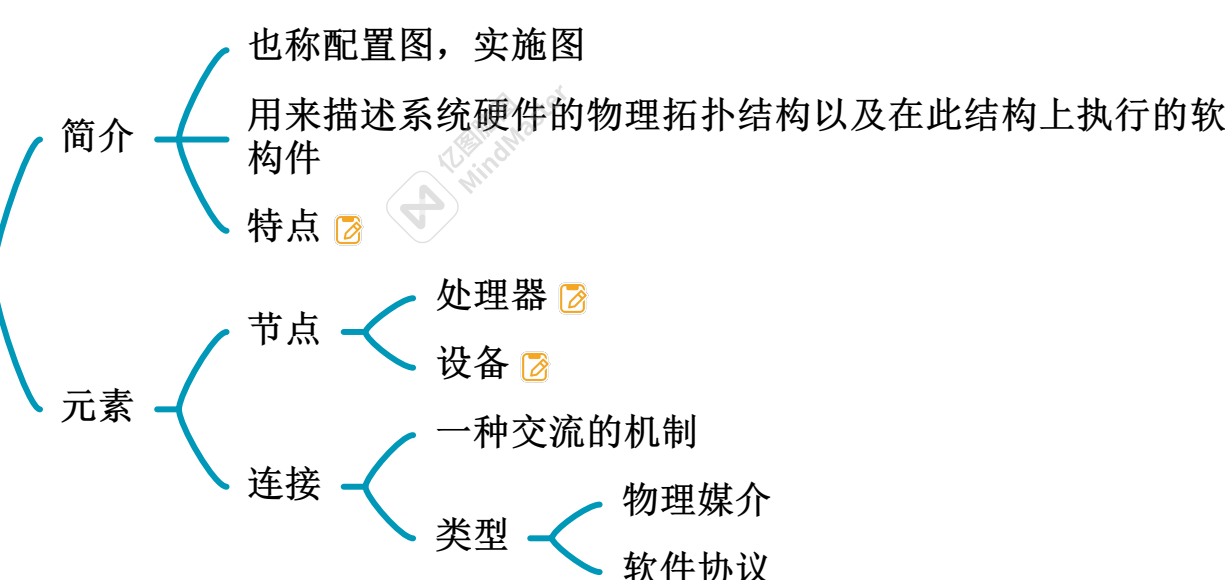
用例图



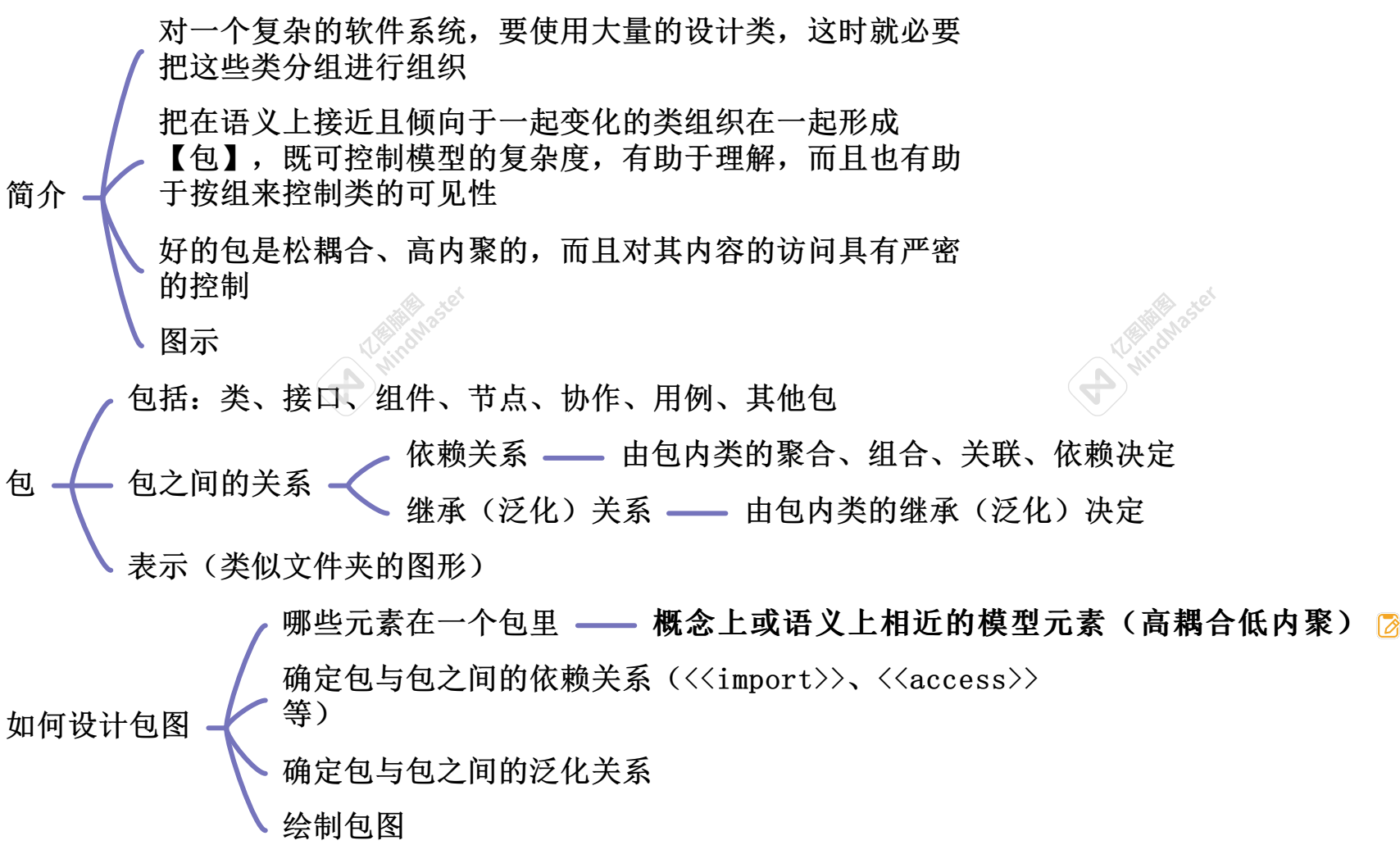
类图



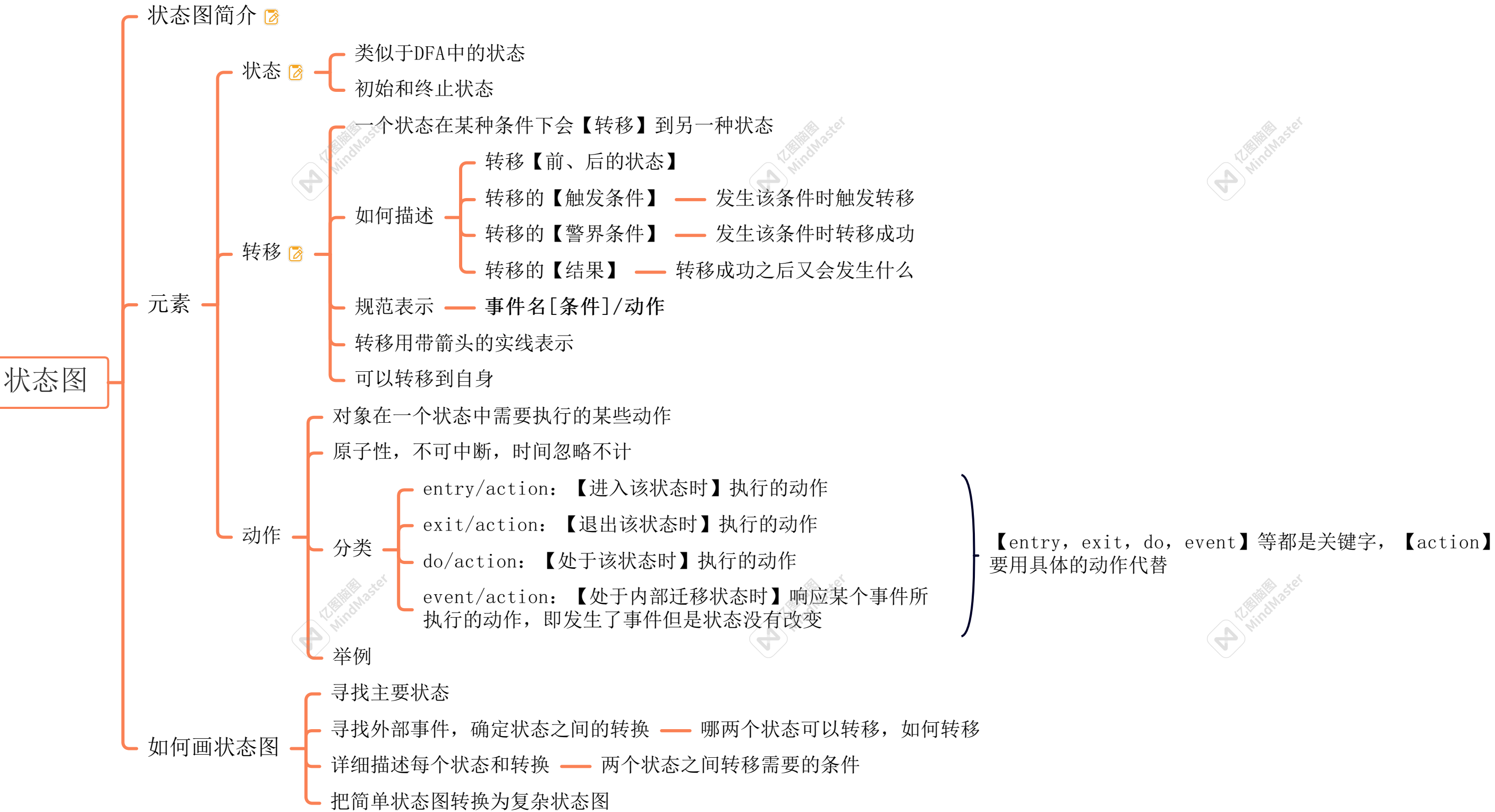
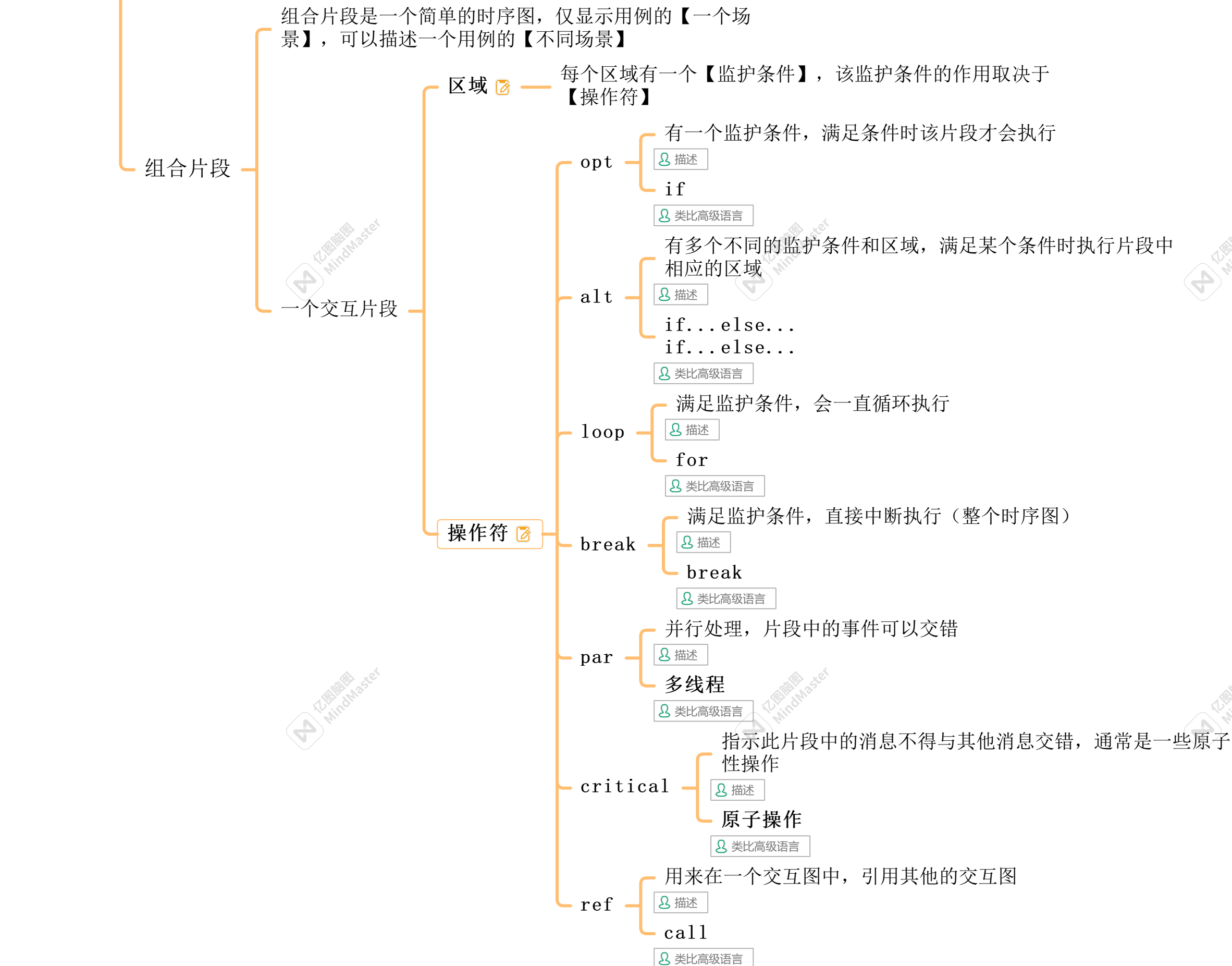
部署图



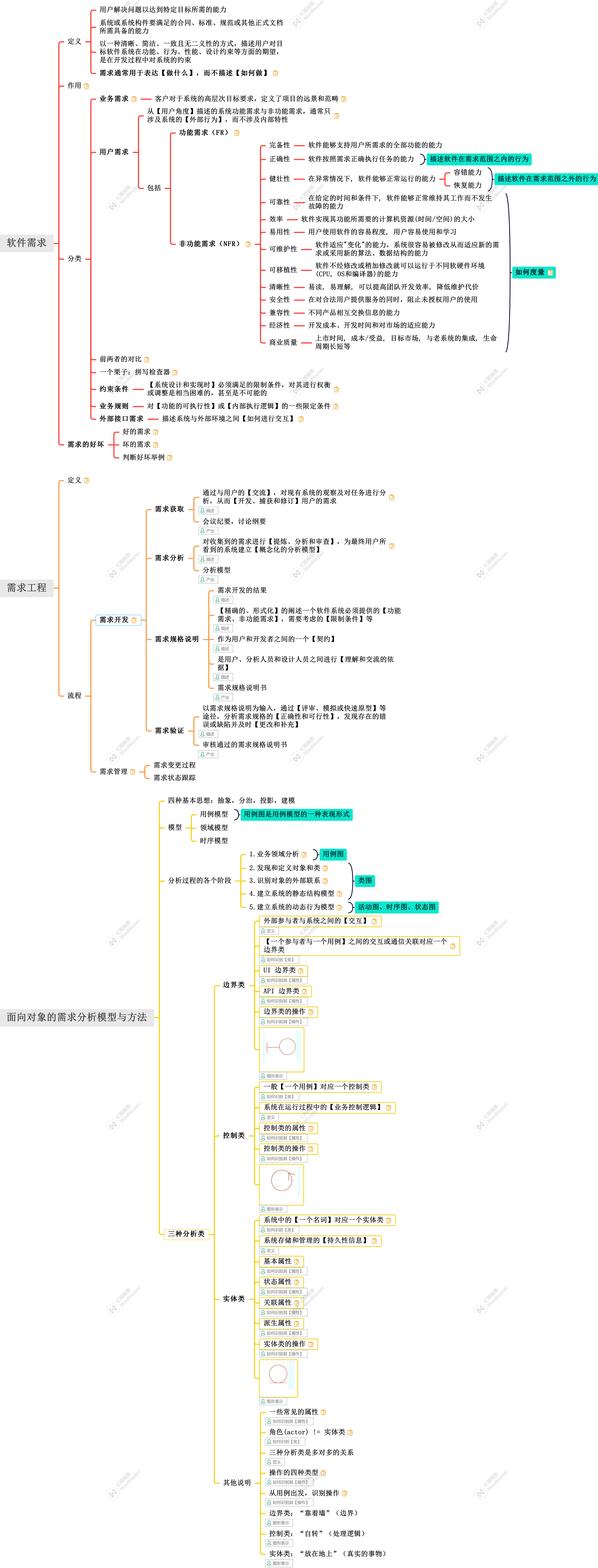
包图



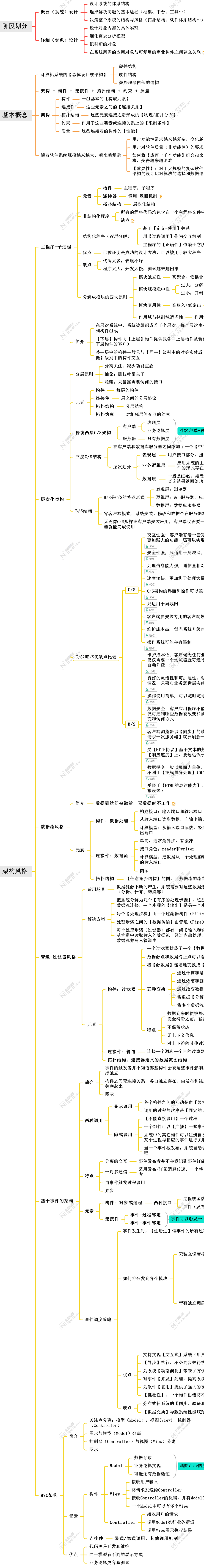
UML (下)



软件需求工程



软件架构风格



本质：将大系统分解为若干模块（模块化），主程序调用这些模块实现完整的系统功能

胖客户端-瘦服务器

C/S

B/S

数据流风格

管道-过滤器风格

基于事件的架构

事件调度策略

优点

缺点

MVC架构

元素

构件

Model

View

Controller

连接件

优点

缺点

自由发挥，积极性提升

因为太随意，所以很乱

每个模块都能收发

软件架构技术

基本模型

- 分层
 - 方式 — C/S, B/S, 多层, MVC
 - 一个模块做很多事情, 各负其责, 分工明确
 - 牺牲了效率, 提升了可维护性
- 异步
 - 方式 — 事件、消息
 - 同步 — 请求之后等待结果
 - 异步 — 后续等待结果
 - 性能(吞吐量)提高, 但实时性变差
- 缓存
 - 一种临时存储, 将数据复制到不同于原始数据源的位置, 访问缓存数据的速度比访问原始数据的速度要快得多, 从而可以减小服务器负载和带宽消耗, 提高用户性能
 - 方式 — 页面缓存、数据缓存、消息缓存
 - 原理 — 从直接到源头去取, 到预取
 - 提高了效率, 但牺牲了准确性
- 并发
 - 方式
 - 集群、负载均衡、分布式数据库
 - 通过消息完成模块之间的功能调用
 - 原理 — 原本一个模块处理很多请求, 使用并发后多个模块共同处理这些请求
 - 提高了吞吐量、响应时间、可靠性, 但需要考虑同步等复杂问题
- 集群
 - 一般集群
 - 集群内各服务器上的内容保持一致
 - 通过并发/冗余提高可靠性与可用性
 - 分布式集群
 - 对业务进行拆分, 集群内各服务器上的内容之和构成系统完整的功能/数据, 每个单独的服务器是对系统功能集合/数据集合的一个划分
 - 通过分布式提高速度与并发性
 - 分布式一定是集群, 集群不一定是分布式

分布式计算架构

基本理论

负载均衡

- 问题引入
 - 单服务器的性能无论如何优化, 总会有一个上限
 - 这是需要多个服务器集群来解决
- 复杂性体现
 - 增加一个任务分配器--【负载均衡器】
 - 为任务选择一个合适的【任务分配算法】
- 负载均衡器
 - DNS 负载均衡
 - 最简单, 地理级别
 - 优点
 - 简单、成本低 — 交给 DNS 服务器处理, 无须自己开发、维护负载均衡设备
 - 就近访问, 提升访问速度 — DNS 解析时可以根据请求来源 IP, 解析成距离用户最近的服务器地址, 可以加快访问速度, 改善性能
 - 缺点
 - 更新不及时 — DNS 缓存的时间比较长, 修改 DNS 配置后, 还是有很多用户会继续访问修改前的 IP, 这样的访问会失败, 达不到负载均衡的目的, 并且也影响用户正常使用业务
 - 扩展性差 — DNS 负载均衡的控制权在域名商那里, 无法根据业务特点针对其做更多的定制化功能和扩展特性
 - 分配策略比较简单 — DNS 负载均衡支持的算法少, 不能区分服务器的差异, 不能根据系统与服务的状态来判断负载, 也无法感知后端服务器的状态
 - 硬件负载均衡
 - 优点
 - 功能强大: 全面支持各层级的负载均衡, 支持全面的负载均衡算法, 支持全局负载均衡
 - 性能强大: 对比一下, 软件负载均衡支持到10 万级并发已经很厉害了, 硬件负载均衡可以支持 100 万以上的并发
 - 稳定性高: 商用硬件负载均衡, 经过了良好的严格测试, 经过大规模使用, 稳定性高
 - 支持安全防护: 硬件均衡设备除具备负载均衡功能外, 还具备防火墙、防DDoS攻击等安全功能
 - 缺点
 - 价格昂贵: 最普通的一台 F5十多万, 好一点近百万元
 - 扩展能力差: 硬件设备, 可以根据业务进行配置, 但无法进行扩展和定制
 - 软件负载均衡
 - 常见的有 Nginx 和 LVS
 - 优点
 - 简单: 无论是部署还是维护都比较简单
 - 便宜: 只要买个 Linux 服务器, 装上软件即可
 - 灵活: 4 层和7层负载均衡可以根据业务进行选择, 也可以根据业务进行比较方便的扩展
 - 缺点
 - 性能一般: 一个 Nginx 大约能支撑 5 万并发
 - 功能没有硬件负载均衡那么强大。
 - 一般不具备防火墙和防 DDos 攻击等安全功能

数据缓存

基本理论

本地缓存

分布式缓存

- 缓存的概念
 - 用于存储数据的硬件或软件组件, 以使得【后续更快访问】响应的数据
 - 缓存中的数据可能是提前计算好的结果、数据的副本等
- 缓存的作用
 - 主要解决高并发, 热点数据访问的性能问题
 - 提供高性能的【数据快速访问】
- 缓存的原理
 - 将数据写入到读取【速度更快】的存储 — 缓存在内存, 硬盘, 或数据库中
 - 将数据缓存到【离应用最近】的位置 — 本地缓存
 - 将数据缓存到【离用户最近】的位置 — CDN缓存, 代理服务器缓存

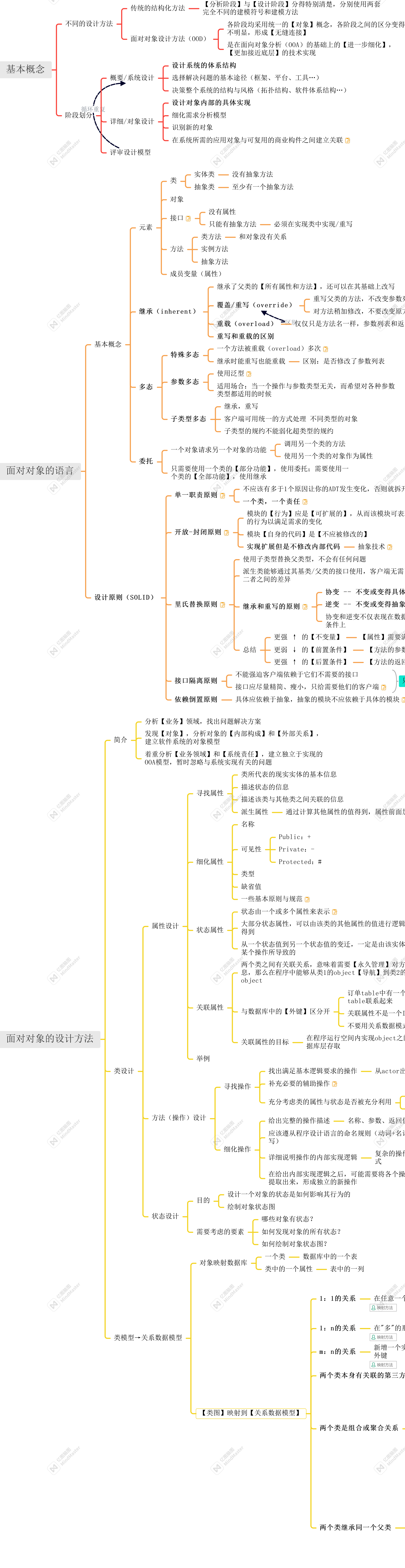
内聚和耦合

内聚

耦合

- 功能性内聚 — 一个模块内所有处理元素【仅完成一个功能】 — 最高程度的内聚
 - 顺序性内聚 — 一个模块内处理元素和【同一个功能密切相关】, 而且这些处理元素必须【顺序执行】 — 数据流从一个【处理单元】流向另一个处理单元
 - 通信性内聚 — 一个模块中所有处理元素都使用【同一个输入数据】或产生【同一个输出数据】
 - 过程性内聚 — 一个模块内的处理元素【是相关的】, 而且必须以【特定的次序执行】 — 控制流从一个【动作】流向另一个动作
 - 时间/瞬时性内聚 — 一个模块包含的任务必须【在同一段时间内】执行
 - 逻辑性内聚 — 模块完成的任务在【逻辑上相同或相似】
 - 偶然性内聚 — 一个模块由完成若干【毫无关系】的功能处理元素【偶然组合】在一起形成 — 最弱的聚合
- 内容耦合 (下列情况之一)
 - 一个模块【直接访问】另一个模块的内部数据
 - 一个模块【不通过正常入口】转到另一模块内部
 - 两个模块有一部分【程序代码重迭】(只可能出现在汇编语言中)
 - 一个模块有【多个入口】 — 最强的耦合, 坚决杜绝使用
 - 公共耦合
 - 若一组模块都访问同一个【公共数据环境】
 - 公共的数据环境可以是【全局数据结构】、【共享的通信区】、【内存的公共覆盖区】等 — 尽量限制使用
 - 外部耦合 — 一组模块都访问同一【全局简单变量】而不是同一全局数据结构, 而且不是通过参数表传递该全局变量的信息
 - 控制耦合 — 如果一个模块通过传送【开关、标志、名字】等控制信息, 【显式地控制选择】另一模块的功能 — 少用
 - 特征/标记耦合
 - 一组模块通过【参数表】传递记录信息, 传递的是数据的地址
 - 这个记录是某一【数据结构的子结构】, 而不是简单变量
 - 被调用的模块可使用的数据【多于它真正需要的数据】 — 都是通过【参数】传递数据
 - 数据耦合
 - 模块之间是通过【简单数据参数】来交换输入、输出信息的
 - 不是控制参数、公共数据结构或外部变量 — 尽量多使用
 - 非直接耦合 — 两个模块之间【没有直接关系】, 它们之间的联系完全是通过【主模块的控制和调用】来实现的 — 最弱的耦合

面向对象的软件设计

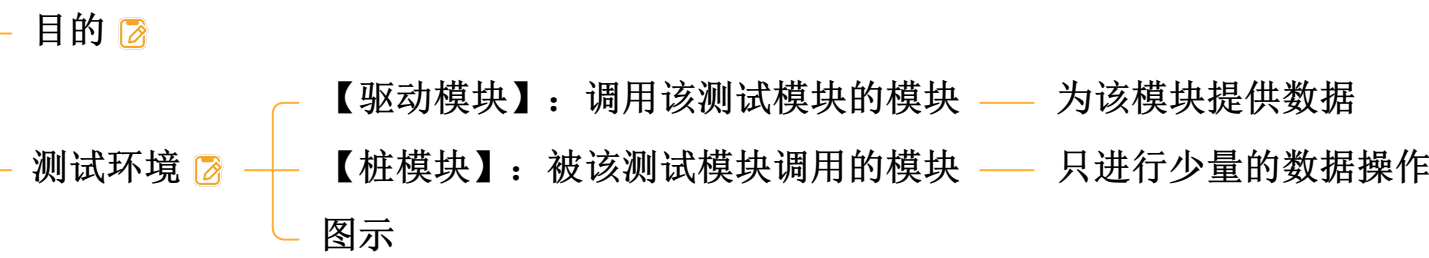


软件测试-概念与方法

基本概念



单元测试

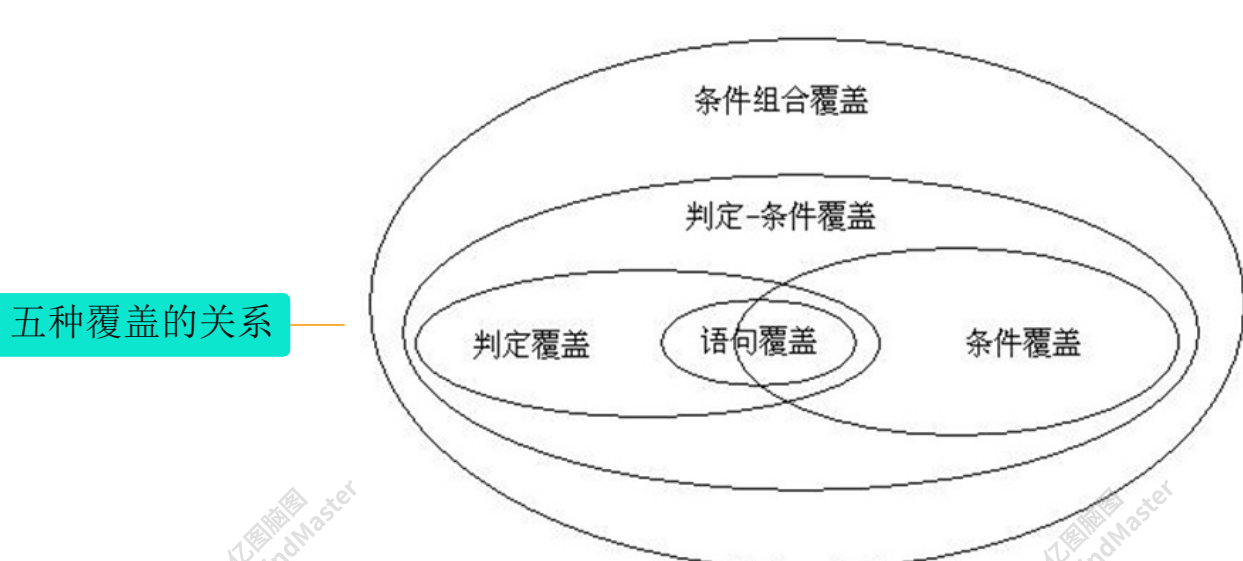
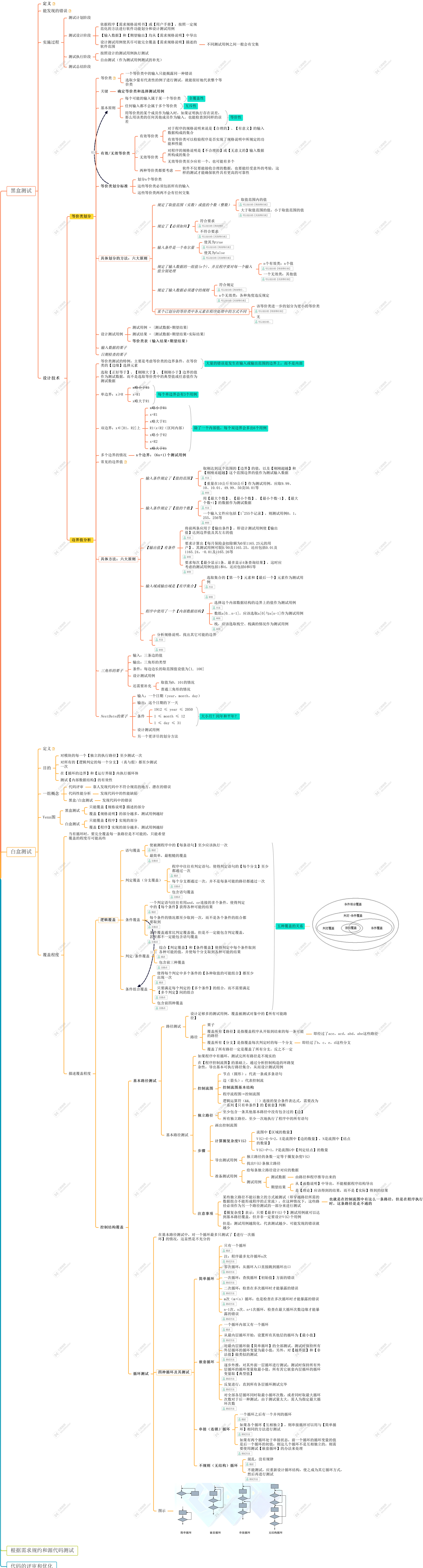


集成测试



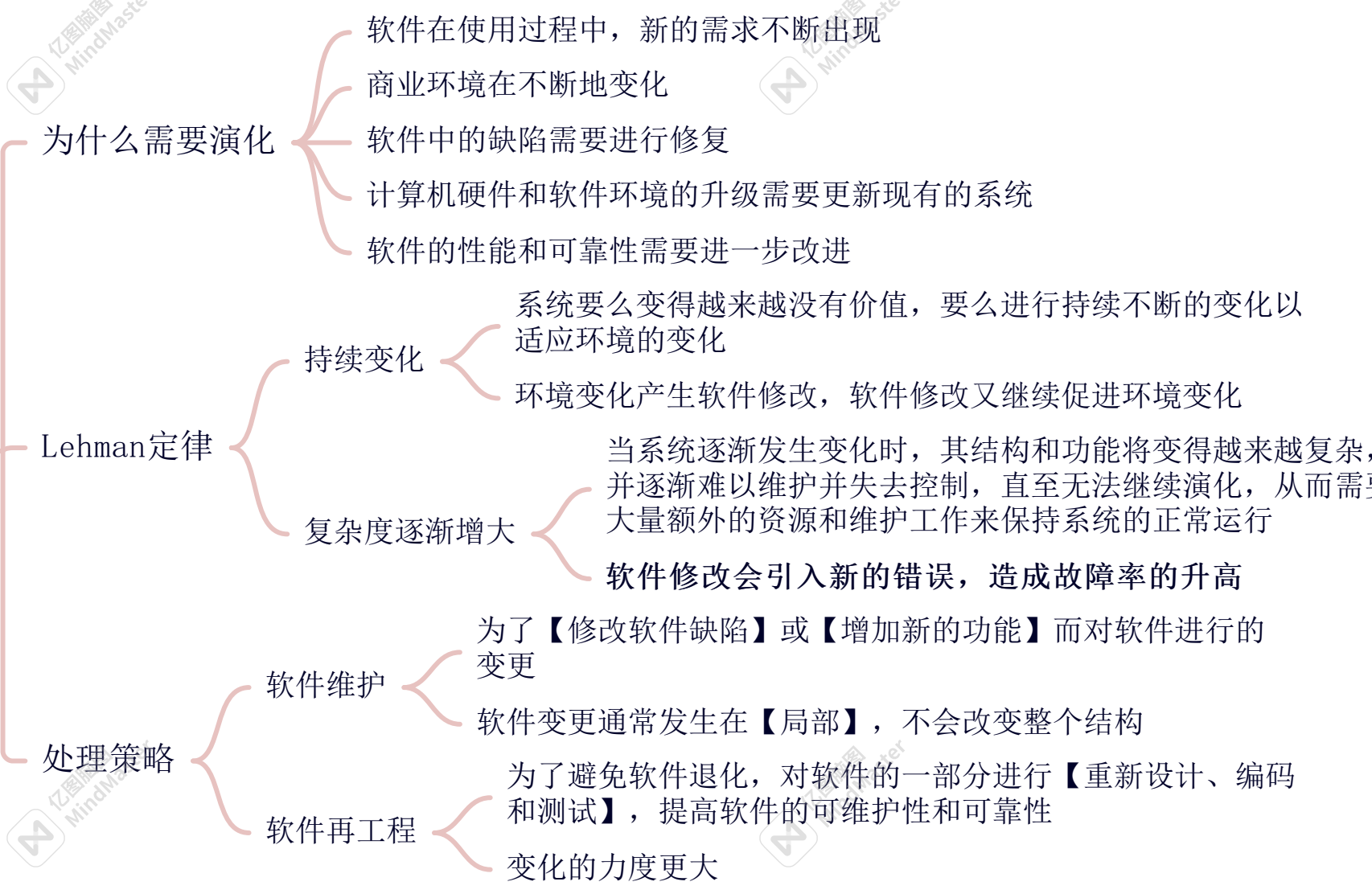
软件测试的类型





软件演化与配置管理

软件演化



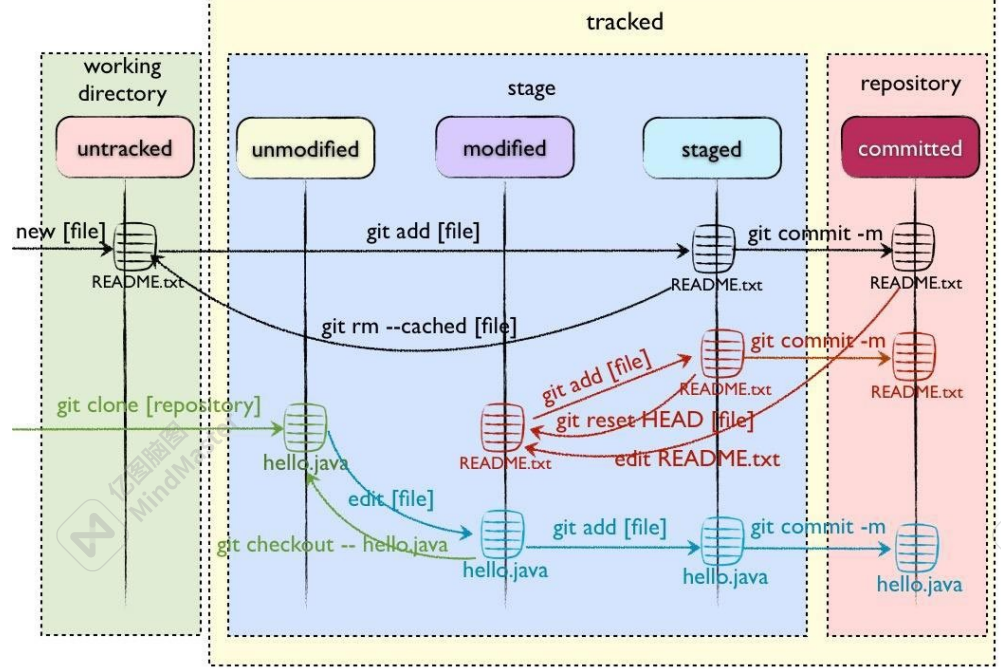
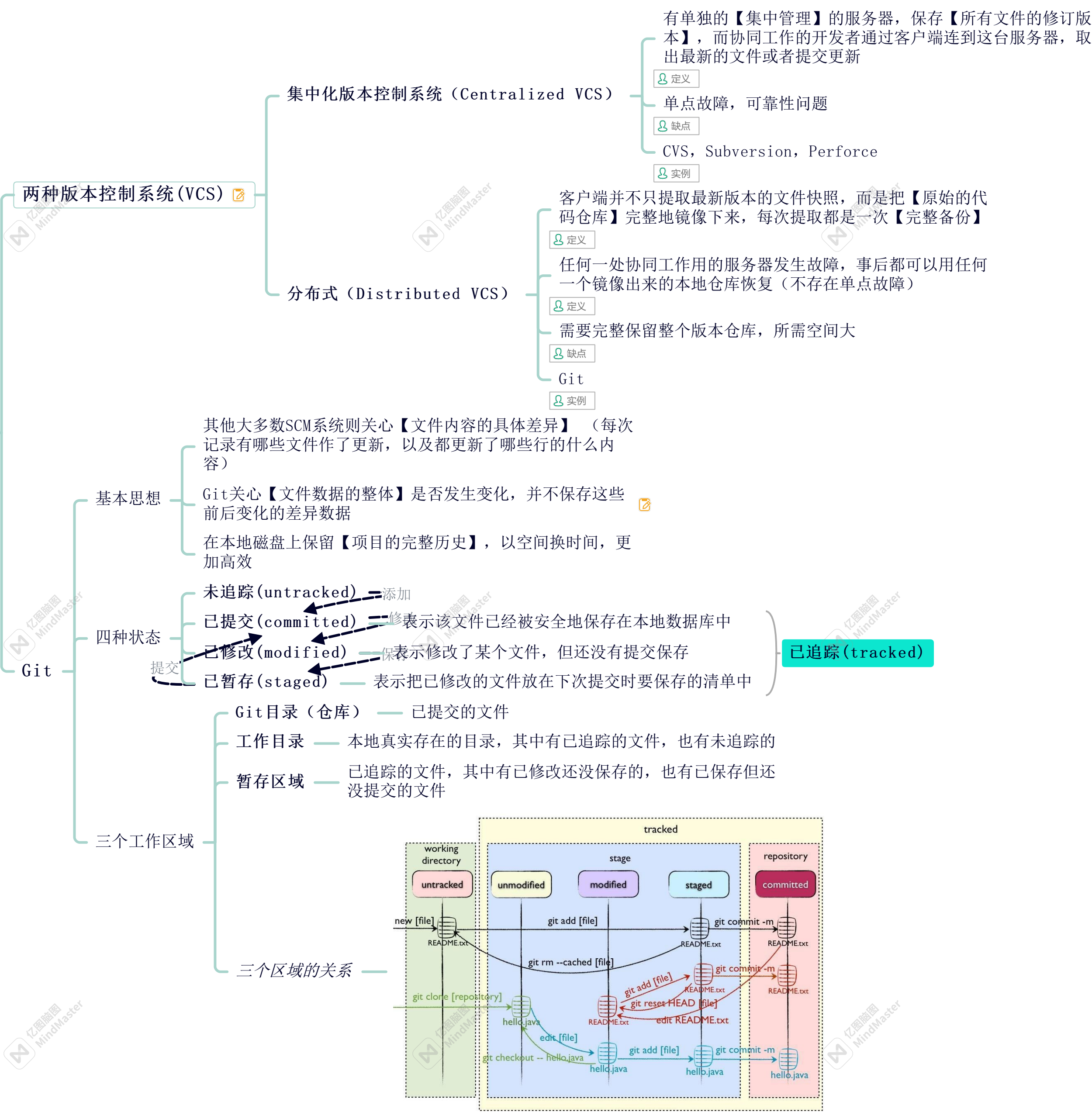
软件维护



软件配置管理(SCM)



版本控制



Git指令



Git指令

